

FI360 MODULE DEVELOPMENT STANDARD

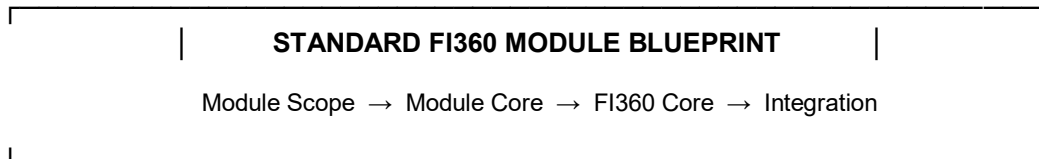
Fleet Intelligence 360 (FI360) — Version 1.0
Fleet Associates Africa Ltd (FAA)
9 August 2026

Document Control

Field	Value
Document ID	FI360-MDS-003
Version	1.0
Status	Baseline / Development Standard
Parent	FI360 Master Architecture & Development Framework v1.1
Technical Foundation	FI360 Core Architecture & Common Services Specification v1.0
Owner	Fleet Associates Africa Ltd
Date	9 August 2026

1. Purpose

This document defines the mandatory development standard for every FI360 business module. It ensures that each module can be developed, tested, deployed and sold independently while remaining structurally compatible with the FI360 Core and other FI360 modules.



2. Development Philosophy

- Build each module as a complete business product, not merely a collection of screens.
- Keep specialist business logic inside the module.
- Use FI360 Core services instead of duplicating identity, tenancy, entitlement, audit and integration infrastructure.
- Expose stable APIs/events for future connectivity.
- Maintain a clear source of truth for every business entity.
- Never create hidden dependencies on another module's database.
- Design for independent licensing and deployment.
- Design integration as an enhancement, not a prerequisite for standalone operation.

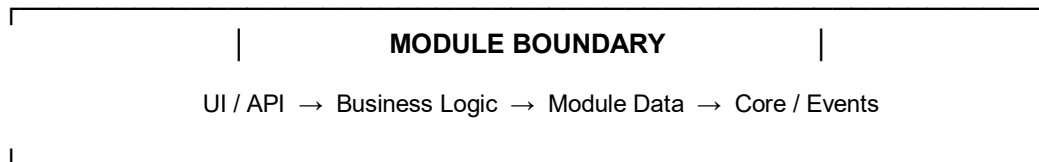
3. Standard Module Structure

Layer	Responsibility
Presentation	Web/mobile UI, navigation, forms, dashboards and user interaction
Application/API	Requests, validation, orchestration and API contracts
Domain/Business Logic	Rules, calculations, workflows and decisions owned

	by module
Data	Module-owned transactional records and repositories
Integration	FI360 Core APIs/events and approved external connectors
Observability	Module logs, metrics, traces and health checks
Security	Module permissions, validation and protected operations

4. Module Boundary Rules

A module owns its domain data and business rules. The FI360 Core owns shared platform capabilities. Cross-module data is accessed through approved APIs/events rather than direct database joins.



FI360 Core	Module
Tenant/user identity	Domain entities
Roles/permissions infrastructure	Domain rules
Module registry/entitlement	Domain calculations
Common references	Domain workflows
Audit infrastructure	Domain reports
API/event infrastructure	Domain transactions

5. Module Identification Standard

Field	Standard
Module ID	Stable unique code, e.g. FI360-TYR
Module Name	Customer-facing product name
Version	Versioned release identifier
API Version	Explicit supported API contract version
Owner	Business/product owner
Technical Owner	Engineering owner
Status	Planned / Development / QA / UAT / Production / Retired
Dependencies	Approved Core/module dependencies
Entitlement	Subscription/activation relationship

6. Standard Module SRS

Every FI360 module must have a dedicated Software Requirements Specification before production development.

SRS Section	Required Content
Purpose	Business problem and objectives
Scope	Included/excluded capabilities
Users	User types and personas
Processes	Business workflows
Functional Requirements	Numbered, testable requirements
Data Requirements	Entities, fields and ownership
UI/UX	Screens, navigation and interaction
Reports	Operational and management outputs

Integrations	APIs, events and external systems
Security	Roles, permissions and sensitive operations
Non-functional	Performance, availability, audit and recovery
Acceptance Criteria	Testable conditions
Assumptions	Known assumptions
Dependencies	Core/module/external dependencies

7. Functional Requirement Standard

Requirements must be uniquely numbered and testable.

ID	Example
FR-TYR-001	The system shall register a tyre against a valid FI360 Asset ID.
FR-TYR-002	The system shall record tyre fitment position.
FR-TYR-003	The system shall prevent unauthorized modification of closed transactions.
FR-TYR-004	The system shall publish the approved TYRE_FITTED event after successful fitment.

8. User Roles and Permissions

Each module shall define its own business permissions while using the FI360 Core authorization model.

Role	Typical Responsibility
Module Administrator	Configuration and controlled administration
Supervisor	Approval and operational oversight
Operator	Daily transaction entry and processing
Viewer	Read-only access
Auditor	Controlled review and audit access
Integration Service	Machine-to-machine operations only

9. Data Model Standard

- Every transactional entity must have a stable unique ID.
- Tenant context must be enforced.
- Created/updated timestamps should be captured.
- Created/updated actor or service should be captured where appropriate.
- Use controlled archival/deletion where historical integrity requires it.
- Cross-module references use canonical IDs and approved interfaces.
- Sensitive fields require explicit classification and protection.

Entity Type	Example
Master	Tyre, Vehicle, Supplier, Work Type
Transaction	Fitment, Inspection, Rotation, Work Order
Reference	Tyre Position, Status, Failure Code
Attachment	Inspection photo/document metadata
Audit	Module action reference

10. UI/UX Standard

- Consistent FI360 navigation and terminology.
- Responsive design for approved device classes.

- Clear create/view/edit/archive patterns.
- Mandatory fields visibly identified.
- Validation before submission.
- Confirmation for high-impact actions.
- Search/filter/sort/pagination for operational lists.
- Consistent status and workflow presentation.
- Accessible error messages and recovery guidance.
- Dashboards distinguish operational from analytical data.

11. API Standard

Every module that exposes functionality to another module or external system must publish a documented API contract.

Requirement	Standard
Authentication	FI360-approved authentication
Authorization	Tenant/module/action permissions
Versioning	Explicit API version
Validation	Schema and business validation
Idempotency	Required for retry-sensitive operations
Errors	Standard FI360 error structure
Pagination	Required for large collections
Tracing	Correlation ID
Documentation	Endpoint, schema, examples and authorization

12. Event Standard

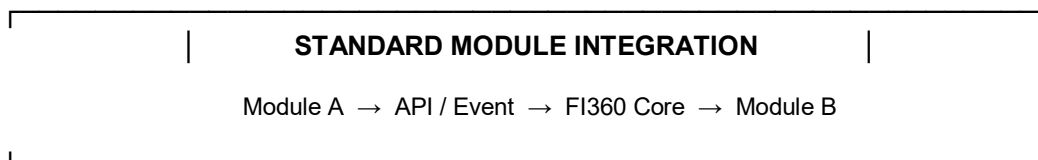
Events communicate business facts, not arbitrary database changes.

Event	Example
TYRE_FITTED	A tyre has been successfully fitted
TYRE_REMOVED	A tyre has been removed
WORK_ORDER_COMPLETED	A work order has been completed
PART_ISSUED	A part has been issued
FUEL_TRANSACTION_POSTED	A fuel transaction has been accepted

- Stable event names.
- Version event schemas.
- Include tenant and entity identifiers.
- Include source and timestamp.
- Include correlation ID where relevant.
- Consumers tolerate duplicate delivery.

13. Integration Pattern

Modules must support standalone operation and controlled integration through FI360 Core interfaces.



1. Module A performs a valid business action.
2. Module A calls a synchronous API or publishes an event.
3. FI360 Core applies security, routing and traceability.
4. Module B consumes approved information.
5. Module B updates its own domain record.
6. Audit and analytics capture the transaction trail.

14. Source-of-Truth Rules

Domain	Authoritative Module
Tenant / User / Role	FI360 Core
Vehicle/Asset Master	Fleet Register
Tyre Lifecycle	Tyre
Work Orders	Workshop
Parts Inventory	Parts
Fuel Transactions	Fuel
Driver Records	Driver
Telematics Device Data	Telematics
Cross-module KPIs	Analytics/Data Platform

A module may cache or reference data owned elsewhere, but it must not silently become a second authoritative source.

15. Workflow Standard

- Define explicit states.
- Define who may perform each transition.
- Define approval rules.
- Record transition history.
- Support rejection/return where applicable.
- Define SLA/escalation requirements where needed.
- Prevent unauthorized state manipulation.

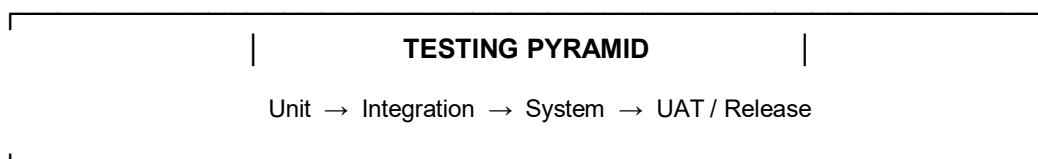
16. Reporting Standard

Report Type	Purpose
Operational	Daily processing and exceptions
Management	KPI and performance monitoring
Analytical	Trends, cost and comparative analysis
Audit	Traceability and control review
Export	Approved data extraction

Reports must use documented metric definitions so that the same KPI has the same meaning across modules.

17. Testing Standard

Testing must occur at multiple levels and should be automated where practical.



Level	Minimum Coverage
Unit	Business rules, calculations and validation
Integration	APIs, events, database and dependencies
System	End-to-end module workflows
Security	Authentication, authorization, tenant isolation
Performance	Critical operational paths
UAT	Customer/business acceptance
Regression	Existing functions after changes

18. Test Case Standard

Field	Required
Test ID	Unique identifier
Requirement ID	Linked FR/NFR
Preconditions	Required setup
Steps	Reproducible actions
Expected Result	Testable outcome
Actual Result	Observed outcome
Status	Pass/Fail/Blocked
Evidence	Screenshot/log/reference where needed

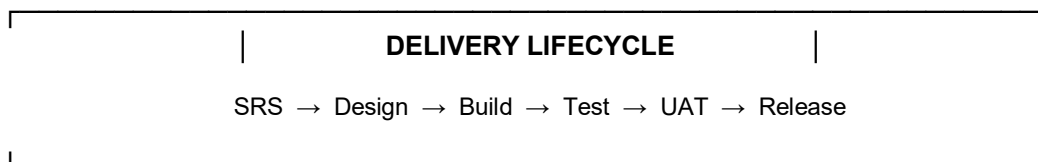
19. Security Standard

- Tenant isolation must be tested.
- Authorization must be enforced server-side.
- Sensitive operations require appropriate permissions.
- Inputs must be validated and sanitized.
- Secrets must not be stored in source code.
- Audit material security and business actions.
- Dependencies must be monitored for vulnerabilities.
- Security defects are classified and tracked before release.

20. Performance and Scalability

- Define performance expectations for critical workflows.
- Identify high-volume transactions.
- Use pagination for large datasets.
- Use asynchronous processing for suitable long-running tasks.
- Avoid unnecessary cross-service calls.
- Measure before optimizing.
- Design stateless services for horizontal scaling where practical.

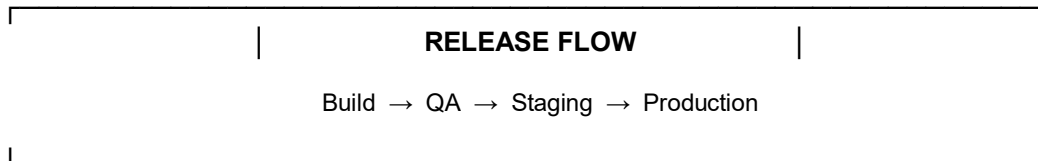
21. Module Delivery Lifecycle



7. Business case and scope.

8. SRS approval.
9. Architecture and UX design.
10. Database/API/event design.
11. Development.
12. Automated and integration testing.
13. UAT.
14. Security and release review.
15. Production deployment.
16. Monitoring and support.
17. Versioned improvement cycle.

22. Deployment and Release



Gate	Minimum Requirement
Build	Versioned source and repeatable build
QA	Automated/integration tests acceptable
Staging	Production-like validation
UAT	Business acceptance completed
Release	Approved change record and rollback plan
Production	Monitoring enabled

23. Versioning

- Use a consistent versioning policy.
- API versions must be explicit.
- Database migrations must be controlled.
- Event schemas must be versioned.
- Breaking changes require migration planning.
- Retired APIs/events require a documented deprecation period.

24. Documentation Package for Each Module

Document	Mandatory
Module SRS	Yes
Architecture Specification	Yes
Data Model / ERD	Yes
API Specification	If APIs exist
Event Catalogue	If events exist
UI/UX Specification	Yes
Test Plan	Yes
Test Cases	Yes
Deployment Runbook	Yes
User Guide	Yes
Support/Operations Runbook	Yes
Release Notes	Yes

25. Developer Backlog Standard

Backlog Item	Required Content
Epic	Business capability
Feature	User/business outcome
User Story	Actor + action + value
Acceptance Criteria	Testable conditions
Technical Task	Implementation work
Bug	Observed defect and evidence
Dependency	External/internal prerequisite
Definition of Done	Completion criteria

26. Definition of Done

- Requirement implemented.
- Code reviewed.
- Unit/integration tests passed.
- Security checks completed.
- API/event documentation updated.
- Database migration documented.
- Audit/observability verified.
- UAT acceptance obtained where applicable.
- Deployment and rollback documented.
- Release notes updated.

27. Module Independence Test

- Module operates without optional business modules.
- Tenant and identity work through FI360 Core.
- Primary workflows function independently.
- Module owns and retrieves its own transactional data.
- Optional integrations can be disabled without destroying core operation.
- Module can be licensed/activated independently.
- Unrelated modules do not need simultaneous deployment unless a documented compatibility dependency exists.

28. Integration Readiness Test

- Canonical IDs identified.
- Source-of-truth entities documented.
- API contracts documented.
- Events documented where required.
- Permissions defined.
- Error/retry behaviour defined.
- Correlation/audit requirements defined.
- Version compatibility defined.
- Integration test cases available.

29. Example: FI360 Tyre Module

FI360 Tyre should be the first reference implementation of this standard. It must operate independently while being integration-ready for Workshop, Fleet, Fuel, Parts, Telematics and Analytics.

Area	Tyre Responsibility
Core dependency	Tenant, identity, authorization, entitlement, audit, APIs/events
Owns	Tyre master/lifecycle, fitment, inspection, rotation, removal, disposition
References	FI360 Asset ID, site, supplier and common reference data
Outputs	Tyre KPIs, lifecycle events and approved API responses
Future connections	Workshop work orders, Parts consumption, Fleet context, Telematics mileage, Analytics/TCO

30. Standard Repository Structure

Area	Example
/docs	SRS, architecture, API, events, runbooks
/src	Application source
/domain	Domain/business logic
/api	API contracts/controllers
/data	Repositories/migrations
/integration	Core/external connectors
/tests	Unit/integration/system tests
/config	Non-secret configuration
/deploy	Deployment definitions
/scripts	Controlled utility scripts

31. Architecture Review Checklist

- Module boundary clear.
- Source of truth documented.
- Dependencies justified.
- FI360 Core used correctly.
- Standalone operation demonstrated.
- APIs/events versioned.
- Tenant isolation enforced.
- Security-sensitive actions audited.
- Acceptance criteria testable.
- Rollback possible.
- Operational metrics and alerts defined.

32. Implementation Sequence for Each New Module

18. Approve business case.
19. Create module SRS.
20. Confirm boundary and source of truth.
21. Define roles and permissions.
22. Define data model/ERD.
23. Define UI/UX and workflows.

24. Define APIs/events.
25. Build using standard repository structure.
26. Execute testing.
27. Complete UAT.
28. Deploy through controlled environments.
29. Monitor, support and release improvements.

33. Acceptance Criteria for This Standard

- Every new FI360 module follows the same development gates.
- Each module can be independently licensed and deployed.
- Each module uses FI360 Core for shared services.
- Each module has explicit data ownership.
- Cross-module integration uses APIs/events rather than database coupling.
- Every module has testable requirements and acceptance criteria.
- Every module has security, audit and observability controls.
- Every module has deployment, rollback and support documentation.

34. Next Document

DOCUMENT 04 — FI360 TYRE MODULE SOFTWARE REQUIREMENTS SPECIFICATION (SRS) should be the first module-specific implementation document. It will apply this standard to the Tyre module and define actual users, workflows, screens, data entities, business rules, reports, APIs, events, permissions, test cases and implementation backlog.

Core principle: Build independently. Integrate deliberately. Share data through governed contracts.